Outbox 与一致性

双写、Transactional Outbox、Saga 与缓存一致性的业务底层

gomall · 业务侧 deck

今日大纲

业务出发点：一致性意味着什么

双写问题：业务上意味着什么

Transactional Outbox：业务变更 + 事件落表

Publisher：轮询 + 状态机推进

至少一次语义：下游必须幂等

协同 vs 编排：Saga 的业务取舍

缓存一致性：双删 + 击穿保护

启动顺序事故：InitLog 必须在 RMQ 之前

一致性是业务事实，不是技术口号

- 业务上的”一致”：库存数、订单状态、搜索结果、用户能看到的页面，四者要对得上。
- gomall 里至少有四个独立存储：MySQL、Redis、RabbitMQ、Elasticsearch。
- 任何一对存储之间都可能”一边成功一边失败”。
- 业务能容忍的不是”零延迟一致”，而是”最终一致 + 看得见的窗口”。

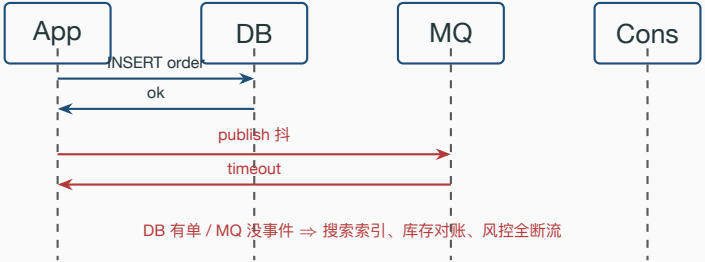
cmd/main.go:29–47 初始化序列里 *4* 个存储依次进场

gomall 里业务侧最在意的三组一致性

一致性对	业务后果	gomall 的兜底	
DB ↔ MQ	下游收不到事件 / 搜索索引腐烂	**Outbox** (deck 主线)	三组
DB ↔ Cache	用户看到旧价 / 旧库存	延迟双删 + SETNX 回源锁	
跨服务步骤	半成品订单、超卖、漏退款	协同式 Saga + 补偿事件	

一致性问题各有典型出错形态。本 deck 只挑这三组，因为 gomall 实际改过、修过、复盘过。*service/order.go*、*service/order_cancel.go*、*service/product.go* 三处对应改动

双写问题的两个不对称失败



反过来：先 publish 再 commit，commit 失败时下游会收到”幽灵订单”，更糟。*docs/blog/04-outbox-saga.md § 这就是双写问题*

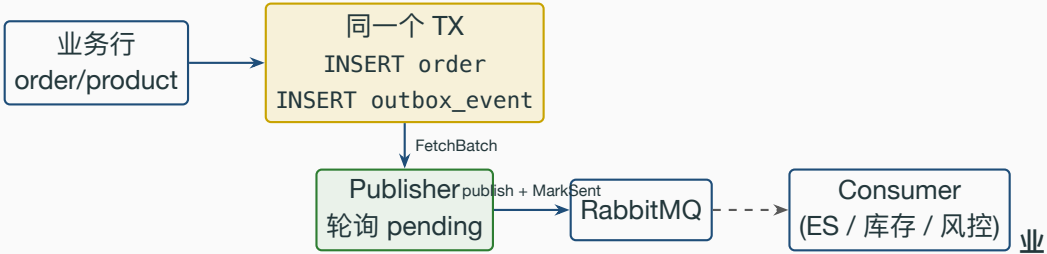
为什么 gomall 不能” 两个 try / catch 套一下”

- DB 事务回滚不会回滚已经发出去的 MQ 消息。
- MQ 投递失败可以重试，但应用进程崩了之后重试上下文也没了。
- 2PC / XA 在 MQ 那一侧支持很差，RabbitMQ 官方没有 XA 资源管理器。
- 业务无法接受” 先停止下单、等运维手动对账”。

业务层结论：必须把” 事件” 也当成 DB 行写进去，让事务统一管。

deck 04 § 双写问题 / 业界标准答案：Transactional Outbox

Outbox 的核心一句话



业务变更 + 事件写在同一个 DB 事务里——要么都成功，要么都失败。

” 投递给 MQ” 这件事被下沉到一个独立的轮询循环，业务请求不再等 MQ ack。

service/order.go::CreateOrder / service/order_cancel.go::CancelUnpaidOrder

outbox_event 表结构：业务字段就是状态机字段

Listing 1: *repository/db/model/outbox.go:17–37*

```go
type OutboxEvent struct {
    gorm.Model
    AggregateType string // "order" / "product" 业务实体类型
    AggregateID uint // 业务实体 id —— 下游幂等 key 之一
    EventType string // "OrderCreated" / "OrderCancelled"
    RoutingKey string // "order.created" 投递到哪条 routing
    Payload string // JSON body —— 包含一切下游需要的字段
    Status int // 1 pending / 2 sent / 3 dead
    Attempts int // 已尝试投递次数
    NextRetryAt time.Time // 指数退避后的下一次允许时间
    LastError string // 最后一次失败原因，截断 500 字
}
```

几个关键设计：**Status** 是业务码（pending/sent/dead），不是布尔；**NextRetryAt** 让退避不依赖外部调度；**LastError** 留给值班同学复盘。

CancelUnpaidOrder：业务变更 + 事件同事务

Listing 2: *service/order_cancel.go:30–51*

```go
err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
    ok, err := dao.NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
    if err != nil {
        return err
    }
    if !ok {
        return nil // 已经被关过，幂等 no-op
    }
    closed = true
    return dao.NewOutboxDaoByDB(tx).Insert(
        "order", "OrderCancelled", "order.cancelled", order.ID,
        events.OrderCancelled{
            OrderID: order.ID, OrderNum: orderNum,
            UserID: order.UserID, ProductID: order.ProductID,
            Num: order.Num, Reason: "timeout",
        },
    )
})
```

Publisher 默认参数：业务为什么这么取

参数	默认值	业务依据
`PollInterval`	**1s**	用户感知” 延迟” 上限 1s 是搜索/对账接受的窗口
`BatchSize`	**100**	单实例 + 1 张表 + 长事务规避，100 行单次能在 50ms 内 commit 完
`MaxAttempts`	**10**	指数退避 1s/2s/4s/.../上限 5min，10 次累计可覆盖约 17 分钟波动

这三个值不是拍脑袋：

`BatchSize=100` 对应” 一次 `FetchBatch` 不会让 publisher 自己变成慢查询”；`MaxAttempts=10` 之后进 dead，由值班看 alarm。*service/outbox/publisher.go:14–32* `withDefaults`；*repository/db/model/outbox.go:14* `OutboxDefaultMaxAttempts`

publisher.loop / drainBatch：主循环 + 状态推进

Listing 3: *service/outbox/publisher.go:51–86*

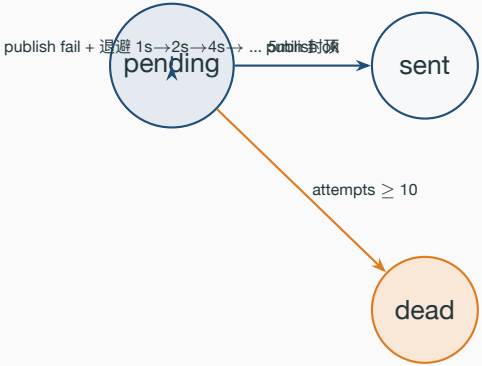
```go
51 func (p *Publisher) loop(ctx context.Context) {
52    ticker := time.NewTicker(p.cfg.PollInterval)
53    defer ticker.Stop()
54    for {
55       select {
56       case <-ctx.Done():
57          return
58       case <-ticker.C:
59          p.drainBatch(ctx)
60       }
61    }
62 }
63
64 func (p *Publisher) drainBatch(ctx context.Context) {
65    if rabbitmq.GlobalRabbitMQ == nil { return }
66    rows, err := dao.NewOutboxDao(ctx).FetchBatch(p.cfg.BatchSize)
67    if err != nil { util.LogrusObj.Errorln(err); return }
68    for _, ev := range rows {
```

publisher.loop 逐行讲：业务侧 6 条要点

1. **`ticker` 而不是 sleep** ——GC / 长 batch 都不会让节奏漂移。
2. **`ctx.Done()`**：进程退出时立刻收尾，正在处理的 batch 让它跑完即可（下次还能 `FetchBatch` 出来）。
3. **`GlobalRabbitMQ == nil` 直接 return**：RMQ 没起，不消耗 DB 也不打 alarm，等运维修。
4. **`FetchBatch`** 拿的是 `pending + next_retry_at <= now()`，自带退避隔离。
5. **失败走 `MarkFailed`** 而不是 `continue` 后丢弃——业务码沿 `pending` 流转，不会”卡死”。
6. **投递成功才 `MarkSent`**：至少一次语义 = 宁可重投不可漏投。

service/outbox/publisher.go:51–86 同上

业务码状态机：pending / sent / dead



- **pending**：默认初值，等下一轮 publisher 取。
- **sent**：业务事件已经送达 broker（注意：不是消费者）。
- **dead**：人工介入分支——值班看 `LastError`，定位下游问题；不会自愈。

repository/db/dao/outbox.go:62–78 `MarkFailed` / *repository/db/model/outbox.go:10–14*

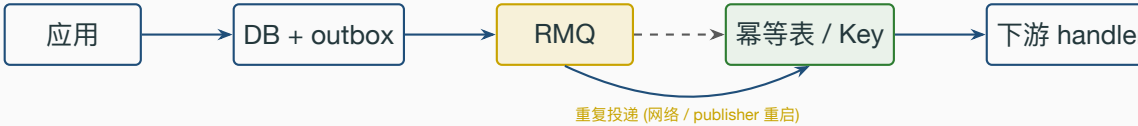
MarkFailed: 指数退避 + 死信落地

Listing 4: *repository/db/dao/outbox.go:62–78*

```go
62 func (d *OutboxDao) MarkFailed(id uint, attempts, maxAttempts int, errMsg string) error {
63    now := time.Now()
64    updates := map[string]any{
65       "attempts": attempts + 1,
66       "last_error": truncate(errMsg, 500),
67       "updated_at": now,
68    }
69    if attempts+1 >= maxAttempts {
70       updates["status"] = model.OutboxStatusDead
71    } else {
72       backoff := time.Duration(1<<attempts) * time.Second
73       if backoff > 5*time.Minute { backoff = 5 * time.Minute }
74       updates["next_retry_at"] = now.Add(backoff)
75    }
76    return d.DB.Model(&model.OutboxEvent{}).Where("id = ?", id).Updates(updates).Error
77 }
```


逐行讲（核心 4 点）：

Outbox 解决” 丢失”，幂等解决” 重复”



- 投递语义：**at-least-once** ——至少一次。
- 把 **at-least-once + 幂等**做出” 业务侧 exactly-once” 的效果。
- 与 deck 01 / 04 / 05 三个 deck 直接衔接：
  - deck 01 幂等中间件：以 `Idempotency-Key` 兜单次请求。
  - deck 04 outbox：本 deck 的技术细节版本。
  - deck 05 限流熔断：客户端重试时再叠加 RateLimit / CircuitBreaker。

压测里量出来的两组真实数字

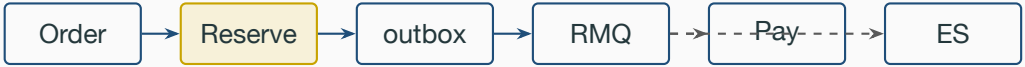
压测场景	
单元测试 `TestCoupon_AtomicClaimNoOversell`：500 并发抢 100 张券	**零**
压测 `idempotency_replay.js`：同 Key 累计 15s 请求	**75**
对应入库订单数（`SELECT COUNT(*)`）	
回放 p95 / 吞吐	2.33ms / **50,319**

” 零超发” + ”75 万次只入 1 单” 是同一套设计的两个表现——**at-least-once 投递**遇上**业务**

码幂等键（券号 / Idempotency-Key），最终对账数永远只算一次。*stressTest/REPORT.md § 单元测试一览 & §4. 幂等中间件压测*

协同 vs 编排：哪种 Saga 配 gomall

模式	实现	业务取舍
编排 (Orchestration)	中心 Coordinator 推	步骤 ≥ 5、需要看板、跨团队
协同 (Choreography)	事件驱动、无中心	**gomall 选用**——步骤少

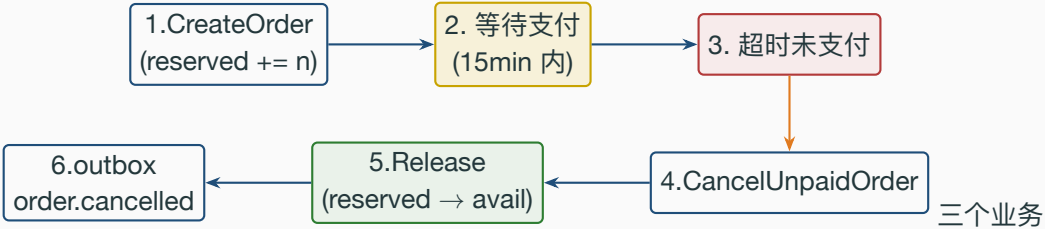


协同式：每一步消费上一步事件、产出下一步事件，没有” 总指挥”。

对 gomall 的 5 步链路 (Order → Reserve → outbox → Pay / ES) 是最省心的选择。

docs/blog/04-outbox-saga.md § 完整链路

Saga 失败一步：CancelUnpaidOrder 就是一次回滚



保证：

- **幂等**：第二次调用 `CloseOrderWithCheck` 返回 `ok=false`，跳过释放与事件。
- **先 DB tx 后 cache**：DB 关单 + outbox 一致后再释放 Redis 占用——失败可重放。
- **最终一致**：库存 = available + reserved + sold 这条不变量始终成立。

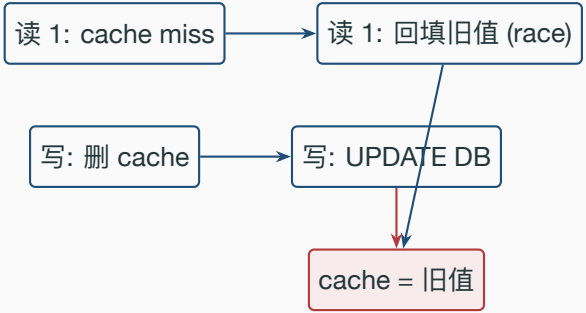
service/order_cancel.go:14–63 完整闭包

业务接受怎样的” 补偿失败”

- Saga 补偿不是银弹：补偿本身也会失败（比如 cache 拒连）。
- gomall 的容错思路：
  1. 先保证 DB 这一边 commit ——业务真相先落地。
  2. Cache 释放失败只打 error 日志，不回滚 DB ——状态机靠下次扫描自愈。
  3. 真出现**长期不一致**时，由 cron + outbox 死信表兜底，值班对账。
- 业务话术（写在 SOP 里）：**” 先关单 + 后释放”，释放失败 = 库存短暂少一点，绝不会卖超。**

service/order_cancel.go:58–62 释放 *reservation* 仅 *log*；*initialize/cron.go* 巡检任务

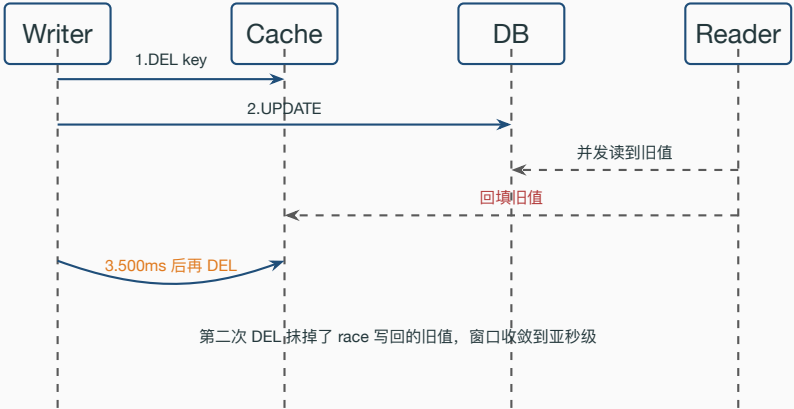
Cache Aside 的两个出错位置



业务出错形态：商品改价后，部分用户在 1～2 秒内还能看到旧价。

对 gomall：**允许**短时间旧库存（最坏拒单），**不允许**长时间旧价格（结算口径不一致）。*service/product.go:282–288 ProductUpdate* 写库前后两次 `DelProductDetail`

延迟双删时序图



repository/cache/product.go:66–74 `DoubleDeleteAsync`

双删 + 击穿保护：源码与逐行讲

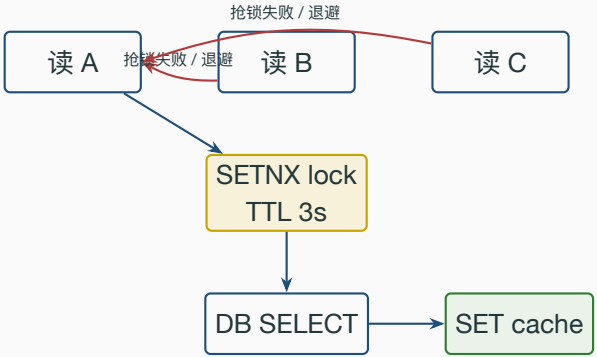
Listing 5: *repository/cache/product.go:55–74*

```go
// TryProductLock SETNX 抢回源锁，避免缓存击穿时多个请求同时回源
func TryProductLock(ctx context.Context, id uint) (bool, error) {
    return RedisClient.SetNX(ctx, ProductLockKey(id), "1", ProductLockTTL).Result()
}

func UnlockProduct(ctx context.Context, id uint) {
    RedisClient.Del(ctx, ProductLockKey(id))
}

// DoubleDeleteAsync 延迟双删：写库后已经做了第一次删除，这里在 interval 后再删一次，
// 用于覆盖"读旧值的并发请求把旧值塞回缓存"的窗口。
func DoubleDeleteAsync(id uint, interval time.Duration) {
    if interval <= 0 { interval = ProductDelayInterval } // 默认 500ms
    go func() {
        time.Sleep(interval)
        _ = RedisClient.Del(context.Background(), ProductDetailKey(id)).Err()
    }()
}
```

击穿保护：SETNX 单飞回源



业务收益：热点商品（首页 banner 商品）失效瞬间不会让 1000 个请求一起打到 DB。

落库次数从 *N* 个请求降到 1 次回源 + 其余请求复用 cache。*repository/cache/product.go:55–62*

`TryProductLock` / `UnlockProduct`

真实事故：`tryInitRabbitMQ` 让进程整个退出

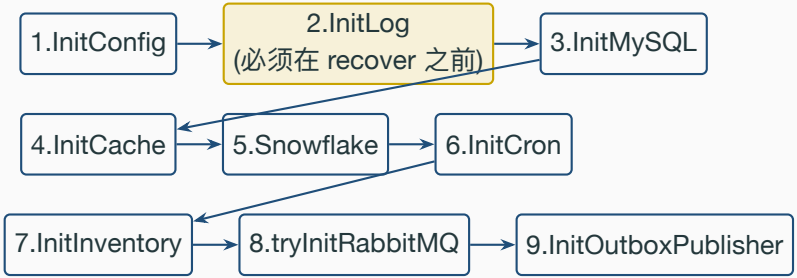
现象：RMQ 没起，`tryInitRabbitMQ()` recover 时 **NPE**，进程 crash。

定位过程：

1. PR #51（订单延迟关单）引入 `tryInitRabbitMQ()`。
2. 该函数 `recover()` 内调用 `util.LogrusObj.Warnf(...)`。
3. 当时 `util.InitLog()` 写在 `tryInitRabbitMQ()` **之后**。
4. `LogrusObj == nil` → recover 内再 panic → 进程退出。

修复（commit 8d0157a / 3cbe3c4）：把 `util.InitLog()` 提前到所有可能 `recover` 的代码之前。*cmd/main.go:30–31 修复后的顺序；stressTest/REPORT.md § 已修：util.LogrusObj*

修复后的启动顺序（业务因果）



关键约束：**InitLog 必须在 InitMySQL / RMQ 之前**，因为 MySQL / RMQ 初始化的 `recover` 都依赖 `LogrusObj`。

cmd/main.go:29–47 完整初始化序列

两条可复用的业务侧教训

1. **初始化顺序**：所有依赖全局对象的代码，必须在它初始化之后才能调用。
   - `InitLog` / `InitTracer` / `InitMetrics` 放最前。
   - 用 `init()` 隐式引导时盯紧 import 顺序。
2. **recover 内禁止再 panic**：
   - recover 是最后兜底，里面再崩进程彻底失控。
   - 写法：recover 内只 `fmt.Fprintln(os.Stderr, ...)` 或写本地文件，不依赖任何复杂组件。

> outbox publisher 起不来不算事故——启动顺序错了让进程 crash，才是事故。

cmd/main.go:54–62 tryInitRabbitMQ 的 recover 块；commit 8d0157a 修复点

业务侧总结：一致性约束一张表

一致性对	模式	业务码 / 表	gomall 现状
DB ↔ MQ	Transactional Outbox	`outbox_event.status`	pending/sent/dead
DB ↔ Cache	Cache Aside + 延迟双删	`product:detail:{id}`	500ms 延迟双删
缓存击穿	SETNX 单飞回源	`product:lock:{id}`	TTL 3s
跨服务 Saga	协同式 (事件驱动)	见 outbox 路由 key	5 步链路
启动依赖	顺序约束	N/A	InitLog 第 2 位

cmd/main.go / repository/cache/product.go / repository/db/dao/outbox.go

面试 Q&A

Q1：为什么 Outbox 比”DB commit 后再 publish” 安全?

A：commit 与 publish 不在一个事务，commit 成功 publish 失败就丢事件。Outbox 把事件写进 DB，由轮询消化，事件丢失等价于 DB 数据丢失。

Q2：outbox 表会不会堆积?

A：会。监控 `status=pending AND next_retry_at <= now()` 的行数即可。横向扩展可用 `SELECT FOR UPDATE SKIP LOCKED`；当前 gomall 单实例 BatchSize=100/1s 还远没到瓶颈。

Q3：延迟双删的” 延迟” 该取多大?

A：取**大于**最长读 race 窗口。gomall 取 500ms：DB 写常态 5–20ms，留 20× 余量盖 GC / 网络抖动；TTL 10min 兜底。

面试 Q&A（续）

Q4：协同式 vs 编排式 Saga，怎么选?

A：步骤 ≤ 5、跨团队少 $\rightarrow$ 协同（gomall 选）。步骤多、需可视化、跨团队多 $\rightarrow$ 编排，但要承担” 中心节点” 运维成本。

Q5：为什么 cache 释放失败不回滚 DB?

A：DB 是业务真相，cache 是性能侧。Cache 失败让库存” 短暂少一点”，最坏拒单不超卖；回滚 DB 反而让” 已关单” 被复活，业务码乱套。

反追问：为什么不用 CDC (Debezium / binlog) 替代 Outbox?

A：CDC 更通用但要拉 binlog 链路、跑 Debezium 集群，运维重；Outbox 是” 应用层 CDC”，简单可控，gomall 量级足够。

代码位置一览

outbox_event 表 *repository/db/model/outbox.go:10–37*

Outbox DAO (Insert / FetchBatch / MarkSent / MarkFailed)

repository/db/dao/outbox.go:13–86

Publisher (Start / loop / drainBatch) *service/outbox/publisher.go:34–86*

初始化 publisher *initialize/outbox.go:12–30*

OrderCreate (业务 + outbox 同事务) *service/order.go:40–102*

CancelUnpaidOrder (Saga 回滚) *service/order_cancel.go:14–63*

缓存双删 / SETNX 回源锁 *repository/cache/product.go:13–74*

ProductUpdate 调用双删 *service/product.go:271–289*

启动顺序修复点 *cmd/main.go:29–47；commit 8d0157a*

配套阅读：deck 04-outbox-saga.tex / deck 03-cache-consistency.tex / docs/blog/04-outbox-saga.md